FDE Case Study 5 — Last-Minute Pushback Prep (30 Q&A)

Nov 10, 2026 · @Adatiya Shaileshkumar

How the round works and how you are scored

The AI asks 4 to 7 questions about your own files and decisions, one at a time, read aloud. You answer by speaking. Each question has a 5-minute timer, and whatever is in the box when it ends gets submitted. At least one question will change a customer requirement and ask how your design copes.

Timing rule. Each answer below is about 450–550 words, which is roughly 4 minutes at a calm speaking pace. Every answer ends with an "If time is left" line that takes you to about 5 minutes. Finish your last sentence before the timer runs out. A cut-off sentence sounds weak.

What the guide asks for. Be specific, name files and decisions, and say plainly what you did not build. Every answer below names the files to say out loud.

Rubric area	Weight	What earns a 5–6
Problem Solving & System Thinking	15%	Start from Elena's rule. Show you found the real problem (the chunker bug) before building.
System Design & Architecture	15%	Give the reason behind each choice and say when you would choose differently.
Customer & Consulting Mindset	15%	Talk about trust, Elena's admins, and what you would confirm with her.
Technical Execution	15%	Give exact numbers, thresholds and file names.
Response to Pushback	15%	Say "you're partly right", then defend with evidence. Never get defensive.
Communication	10%	Use the same structure every time, plain words, and a clear ending.

Rubric area	Weight	What earns a 5–6
Debugging	10%	Describe how you found a bug, what the evidence was, and what you changed.
Product & Business Acumen	5%	Talk about cost, the gap report, rollout and success metrics.

Answer template (use it every time).

1. **Direct answer** in one or two sentences.
2. **Why**: the reason, with a number or an example.
3. **Where in the code**: name one to three files.
4. **Honest limit**: what it does not handle yet.
5. **Next step**: what you would do in production.
6. **Closing line**: one sentence that ties it back to Elena's rule.

Phrases to avoid, and what to say instead

- "100% accuracy" → "42 out of 42 on a small set I wrote; strong evidence, not proof."
- "Surrender" → "refuse" or "honestly say I don't have an answer."
- "It's in the solution doc, we can see later" → say the point now.
- "We will improve it again and again" → name the exact next step.
- "The LLM handles it" → say which layer handles it and how the code checks it.

Cheat sheet: numbers and files to have ready

Read this table once more just before you start. Almost every answer uses these numbers.

What	Value
Corpus	6 articles (D-01 to D-06), 12 chunks of ~400 characters with a 1-sentence overlap
Starter-kit chunker bug	`len(text)//400` drops each article's tail; D-02 and D-05 get zero chunks; Q-2 (15 minutes), Q-3 (10,000 rows) and Q-4 (60 requests per minute) are lost

What	Value
Starter-kit score bug	Q-5 Salesforce (not covered) scores 0.036, above Q-1 password reset (covered) at 0.034
Layer 1 thresholds	LLM mode: term coverage ⩾ 0.2 OR cosine ⩾ 0.35. Extractive mode: coverage ⩾ 0.6, and cosine cannot pass
Retrieval	BM25 (k1 = 1.5, b = 0.75) + all-MiniLM-L6-v2 embeddings, fused with RRF (constant 60), top 4 chunks
LLM settings	Groq openai/gpt-oss-20b, temperature 0, JSON mode, max 1,500 output tokens, 20 s timeout, 1 retry after 0.5 s, auth errors not retried
Follow-ups	Last 3 turns; earlier answers cut to 300 characters; question rewritten and shown as "Searched for: …"
Starter kit results	Dev 4/25, test 6/13, answered all 15 unanswerable questions
Extractive v1 / LLM v1	17/25 / 24/25 (single 25-question set, no split)
Extractive v2	Dev 25/25, test 9/13, 2/15 unanswerable answered, 2/23 false refusals, p50 34 ms
LLM v2	42/42 in each of 3 runs, 0/16 unanswerable answered, 0/26 false refusals, p50 505–584 ms, max 1,215 ms
Tests	52 tests, pass on Python 3.9 and 3.12, no network or key
Held-out misses (extractive)	"minimum length", "protocol" (unseen words); "how many members can I invite?" (limit not in docs)

Files to name

Topic	File
Main flow	`app/services/question_answering_service.py`

Topic	File
LLM mode, retry, fail-safe	`app/services/answering/llm_answer_strategy.py`
No-LLM mode	`app/services/answering/extractive_answer_strategy.py`
Layer 1	`app/services/guards/relevance_gate.py`
Layer 3	`app/services/guards/grounding_verifier.py`
Partial check (extractive)	`app/services/guards/entity_coverage_checker.py`
Layer 2 prompt	`app/llm/prompts/answer_system_prompt_v1.txt`
Chunking fix	`app/ingestion/sentence_chunker.py`
Search	`app/retrieval/hybrid_retriever.py`, `app/retrieval/bm25_index.py`, `app/retrieval/glossary.py`
Follow-ups	`app/services/followup_rewriter.py`
Gap report	`app/services/insights_service.py`, `app/dao/query_log_dao.py`
LLM providers	`app/llm/llm_client_factory.py`, `app/llm/groq_llm_client.py`, `app/llm/openai_compatible_llm_client.py`
Config	`app/core/settings.py`, `app/core/component_registry.py`
Errors and logs	`app/core/exceptions.py`, `app/core/exception_handlers.py`, `app/core/middleware.py`
Redaction	`app/utils/redaction.py`
Eval and tests	`eval/run_eval.py`, `data/eval_questions.json`, `tests/unit/test_answer_strategies.py`, `tests/unit/test_guards_and_utils.py`, `tests/unit/test_chunking.py`

Part A: Problem and walkthrough (Q1–Q4)

These questions test whether you understood the real problem before you started building. Lead with Elena's rule, not with technology.

Q1. Walk me through the problem and what you built.

Rubric focus: Problem Solving, Communication.

Answer: Elena Torres runs support at Solstice Learning. New admins keep asking the same questions about password resets, export limits and billing, and the answers are already in the docs. She wants an admin to ask in plain English and get a real answer from the documentation. Her one hard rule is that if the docs don't cover something, the assistant must say "I don't have an answer for that" and not invent one. I treated this as a trust problem. A wrong answer costs more than an honest "I don't know", because admins act on what the tool says.

Before building anything, I ran the starter kit against the seed data. Its chunker uses `len(text)//400`, so it drops the end of every article. Two articles, D-02 and D-05, got no chunks at all. The facts that answer three of the five sample questions were in that lost text: 15 minutes for reset links, 10,000 rows for CSV export, and 60 requests per minute for the Growth API. So the first fix was retrieval. In `app/ingestion/sentence_chunker.py` I split articles into sentences with citable IDs like D-01.S6 and pack them into overlapping chunks, so nothing is lost. A test in `tests/unit/test_chunking.py` proves it.

On top of retrieval I built three layers against made-up answers. Layer 1, in `relevance_gate.py`, is a cheap code check that refuses obvious misses before any LLM call. Layer 2 is the LLM. It reads numbered sentences and returns JSON saying answered, partial or not_covered, with the sentence IDs it used. Layer 3, in `grounding_verifier.py`, is plain code again. It checks that every citation was in the context we gave, and that every number in the answer appears in the cited sentences. If any check fails, the draft is thrown away and the admin gets an honest refusal.

I also built a no-LLM extractive mode that only quotes sentences, follow-up questions, and a report of unanswered questions for the docs team.

Results: the starter kit scored 4 out of 25 on my dev questions and answered every unanswerable question. My LLM mode scored 42 out of 42 in each of three runs and answered none of the 16 unanswerable ones. That set is small and I wrote it myself, so the next step is testing on real questions from Elena's tickets.

Files to name: `app/ingestion/sentence_chunker.py`, `app/services/guards/relevance_gate.py`, `app/services/guards/grounding_verifier.py`, `app/services/answering/llm_answer_strategy.py`.

If time is left: Mention the fail-safe. If the LLM call fails, the code retries once and then says "I can't verify an answer right now", so the rule holds even on a bad day.

If they push back ("isn't 42/42 overfitting?"): Agree it is a risk. Point to the extractive drop from 25/25 on dev to 9/13 on held-out as proof that you measured it rather than hid it.

Q2. What does "I don't have an answer" mean technically? When exactly does your system refuse?

Rubric focus: System Thinking, Customer Mindset.

Answer: For me, Elena's rule became a precision-first design with three possible outcomes, not two. "Answered" means the docs fully answer the question. "Partial" means the docs answer part of it, and we say what is missing. "Not covered" means the docs don't answer it, and we say "I don't have an answer for that in the documentation" and show related articles. I added partial because many real questions sit in between. For example, "Is SSO available on the Growth plan?" The docs talk about SSO, but only for Enterprise. Saying "yes" would be wrong, and a flat "I don't know" hides useful information.

The system refuses in four places. First, in `relevance_gate.py`, if too few of the question's important words appear in the best chunk and the embedding similarity is also low, we refuse without calling the LLM. Second, if the LLM returns not_covered, or any status we don't expect, we refuse. Third, in `grounding_verifier.py`, if the answer is empty, has no citation, cites a sentence we never gave it, or contains a number that is not in the cited sentences, we discard it and refuse. Fourth, if the LLM call fails after one retry, we say "I can't verify an answer right now". That message is different on purpose, because it tells the admin the problem is ours and not a gap in the docs.

Every refusal is logged with the question redacted. `app/services/insights_service.py` groups the refused questions by how often they come up, at `/api/v1/insights/documentation-gaps`. So a refusal is not a dead end. It becomes a to-do list for Elena's docs team.

The response also carries a `decision_reason` field, for example `grounding_check_failed`. That lets us see exactly which layer refused and why.

Files to name: `app/services/guards/relevance_gate.py`, `app/services/guards/grounding_verifier.py`, `app/services/answering/llm_answer_strategy.py`, `app/services/insights_service.py`.

If time is left: Say the threshold choice leans toward refusing. When unsure, the system says it doesn't know, because that matches Elena's priority.

If they push back ("won't it refuse too much?"): In LLM mode it falsely refused 0 of 26 answerable questions. Extractive mode falsely refused 2 of 23. In production I would watch the refusal rate weekly.

Q3. What did you find in the starter kit, and how did it change your plan?

Rubric focus: Debugging, Problem Solving.

Answer: I found three problems, and they changed my order of work. I fixed retrieval first, instead of starting with the LLM.

The first problem was the chunker. `chunk_text` computes the number of chunks with `len(text)//400`. Integer division rounds down, so the last part of every article is dropped. For short articles like D-02 and D-05 the result is zero, so they produce no chunks at all. When I checked which facts were lost, they were exactly the answers to Q-2, Q-3 and Q-4. No prompt and no LLM can fix that, because the answer never reaches the model.

The second problem was the score. The starter kit uses a Jaccard keyword overlap. Q-5 about Salesforce, which the docs don't cover, scored 0.036. Q-1 about password reset, which the docs do cover, scored 0.034. The uncovered question scored higher than the covered one, so no single threshold can separate them. That told me refusal cannot depend on one similarity score. That is why I built three layers.

The third problem was the evaluation. It only checked whether the right document ID came back. The starter kit could "find" D-01 while the fact itself had been cut off. So my eval in `eval/run_eval.py` checks three things: the status, the cited documents, and a key phrase in the answer.

I kept the starter code unchanged in the `baseline/` folder, so `python -m eval.run_eval --baseline` shows the before and after on the same questions. The starter kit scored 4 out of 25 on dev and answered all 15 unanswerable questions.

Files to name: `baseline/docs_qa_utils.py`, `app/ingestion/sentence_chunker.py`, `tests/unit/test_chunking.py`, `eval/run_eval.py`.

If time is left: Explain the client angle. I would tell Elena early, because her team may already use this chunker logic somewhere else.

If they push back ("why not just fix the division?"): Rounding up would keep the text, but fixed-size chunks still cut sentences in half. Sentence chunks give clean citable IDs, which Layer 3 needs.

Q4. Which design decision are you most confident about, and which one least?

Rubric focus: System Design, Response to Pushback.

Answer: The decision I am most confident about is the code-level grounding check in `grounding_verifier.py`, which is Layer 3. It doesn't trust the LLM. After the model answers, plain code checks that each cited sentence ID was really in the context we sent, and that every number in the answer appears in those cited sentences. If not, the answer is discarded. This matters because Elena's promise should not depend on a prompt being obeyed. Models change. We might move from Groq to Azure OpenAI, or to a cheaper model next quarter, and prompts behave differently across models. A deterministic check keeps working whatever model sits behind it. I proved it with tests in `tests/unit/test_answer_strategies.py`, using a scripted fake LLM that invents "24 hours", invents a citation D-99.S1, or gives no citation. In every case the system refuses.

The decision I am least confident about is the 0.35 cosine threshold in Layer 1. It is a judgement value for the all-MiniLM-L6-v2 model, and it is the least validated number in the system. I picked it to be lenient, because in LLM mode Layer 2 does the real judgement and Layer 1 only filters obvious misses. But I never tuned it on a big set. If the docs grow or the embedding model changes, that number could let in too much or block too much.

The second weak spot is that Layer 3 checks citations and numbers, not every word. If the LLM changes "must" to "may", or drops a "not", my check won't catch it. Words like "two" or "twice" are not checked either, because the number check compares digits only.

What I would do: tune the cosine threshold on real ticket questions, and add an entailment (NLI) check that tests whether the cited sentences actually support the answer sentence by sentence.

Files to name: `app/services/guards/grounding_verifier.py`, `app/services/guards/relevance_gate.py`, `tests/unit/test_answer_strategies.py`.

If time is left: Being clear about weak spots is itself part of the trust story with a client.

If they push back ("so why not remove the cosine path?"): Without it, paraphrased questions that share few exact words would be refused. Coverage alone is too strict in LLM mode.

Part B: Design and architecture pushback (Q5–Q14)

These questions challenge your choices. Start with "you're partly right" where it is true, then defend with evidence, and say when you would choose differently.

Q5. Six docs fit in the context window. Why not put them all in the prompt and skip retrieval?

Rubric focus: System Design, Response to Pushback.

Answer: You're partly right. For six articles, putting everything in the prompt is a valid option, and it would work today. I listed it in my design document as one of the alternatives I considered. It has real strengths: no retrieval misses and simpler code.

I chose retrieval for two reasons.

The first and bigger reason is verification. Retrieval gives me a small set of numbered sentences, around four chunks. My code in `grounding_verifier.py` can then check that every citation the LLM gives is one of those exact sentences, and that every number in the answer appears in them. With the whole corpus in the prompt, the model can pull from anywhere, the context is large, and checking that the answer really comes from the right place becomes much weaker. Elena's rule depends on that check.

The second reason is growth. My assumption, which I'd confirm with Elena, is that the knowledge base grows from 6 articles to hundreds, updated weekly. At that size the whole corpus no longer fits, cost per question rises with every article, and latency goes up. I would rather not rebuild the design when that happens. With retrieval, the cost per question stays about the same as the docs grow.

There is also a smaller cost point. Layer 1 in `relevance_gate.py` refuses obvious misses, like a weather question, without calling the LLM at all. With long context, every question pays for the full prompt.

To be fair to the other side, prompt caching would reduce the cost of long context a lot. So cost alone is not my main argument at six documents. Verification and growth are.

When would I switch? If Elena told me the docs will stay small, say under about 50 pages, and cost is fine, long context with the same Layer 3 check on top would be reasonable. The part I would never drop is the code-level verification.

Files to name: `app/retrieval/hybrid_retriever.py`, `app/services/guards/grounding_verifier.py`, `app/services/guards/relevance_gate.py`.

If time is left: Retrieval also gives article and sentence IDs that the UI shows to the admin as quoted sources. That builds trust in a way a long-context answer cannot.

If they push back ("retrieval can miss the right chunk"): True. That's why I use hybrid search and a glossary, and why a miss leads to a refusal rather than a wrong answer.

Q6. How did you choose 0.2, 0.35 and 0.6? Aren't they just tuned to your 25 dev questions?

Rubric focus: Technical Execution, Response to Pushback.

Answer: Partly yes, and I measured that risk instead of hiding it. Let me explain each number.

The 0.6 term-coverage threshold is for extractive mode. Term coverage means the share of the question's important words, after removing stopwords and stemming, that appear in the best chunk. Extractive mode has no LLM to judge meaning, so Layer 1 is its main protection. I tuned 0.6 on the dev set. Lower values let partial questions through. Higher values refused paraphrases. In this mode cosine similarity is not allowed to pass the gate, because cosine alone is too loose without an LLM behind it.

The 0.2 threshold is for LLM mode, and it is deliberately lenient. It means at least one important word of a five-word question must appear. In LLM mode, Layer 1 is only a cheap filter for obvious misses. The real judgement happens in Layer 2. If I made Layer 1 strict here, it would refuse good paraphrased questions before the LLM ever saw them.

The 0.35 cosine threshold is a judgement value for the all-MiniLM-L6-v2 embedding model. It lets a question pass when it means the same thing but uses different words. It is the least validated number in my system, and I say that in the design document.

To check for overfitting, I wrote 13 held-out test questions before tuning and never tuned on them. The result shows the risk clearly. Extractive mode went from 25/25 on dev to 9/13 on held-out. The misses were unseen words like "minimum length" and "protocol", and a limit the docs don't state. LLM mode held at 13/13, because Layer 2 handles meaning.

When the docs change, these numbers should be re-checked. The plan is a labelled set of real ticket questions, run with `eval/run_eval.py` in CI on every change, with refusal rate and false refusals tracked per threshold.

Files to name: `app/services/guards/relevance_gate.py`, `app/core/settings.py`, `eval/run_eval.py`, `data/eval_questions.json`.

If time is left: The thresholds are settings, not hard-coded, so they can be changed per client without a code change.

If they push back ("so your extractive mode is overfitted"): Yes, somewhat. That is exactly why I recommend LLM mode, and why I report the held-out score next to the dev score.

Q7. Why hybrid search with BM25 and embeddings? Why not embeddings only?

Rubric focus: System Design, Technical Execution.

Answer: Because the two methods fail in different ways, and admin questions need both.

BM25 is keyword search. It is very good at exact terms, like "CSV", "SSO", "429" or a plan name like "Growth". These exact words matter a lot in product docs, and embeddings often blur them. For example, embeddings may treat "Growth" and "Enterprise" as similar, because both are plan names. BM25 is also explainable. I wrote it in about 50 lines in `app/retrieval/bm25_index.py`, with the standard settings k1 = 1.5 and b = 0.75, so I can see exactly why a chunk scored high. I also reuse its IDF weights to rank sentences in extractive mode.

Embeddings, using all-MiniLM-L6-v2, catch meaning when the words differ, like "I can't log in" versus "forgot password".

To combine them, I use Reciprocal Rank Fusion in `app/retrieval/hybrid_retriever.py`, with the standard constant 60. RRF combines the rank positions, not the raw scores. That matters because BM25 scores and cosine scores are on completely different scales, so adding them directly would be wrong.

On top of that there is a glossary in `data/glossary.json`, maintained by support. It maps admin words to doc words, like "add a user" to "invite". It only changes the search text. The LLM always sees the admin's own wording.

Embeddings are optional on purpose. They need a large download, and my Docker image with them was about 2.8 GB. Without them, the system runs keyword-only and the tests still pass.

We return the top 4 chunks. With 12 chunks that is a third of the corpus, which is fine now but must shrink as the corpus grows.

Files to name: `app/retrieval/hybrid_retriever.py`, `app/retrieval/bm25_index.py`, `app/retrieval/embedding_model.py`, `app/retrieval/glossary.py`.

If time is left: At scale I'd add a cross-encoder reranker after hybrid search, because cosine is a weak, uncalibrated relevance signal.

If they push back ("why not rank_bm25?"): It would work. I wanted every score visible and the IDF reusable, and 50 lines was cheap.

Q8. Why sentence-level chunks with IDs? Why around 400 characters?

Rubric focus: Technical Execution, System Thinking.

Answer: Sentence IDs are what make the answer checkable.

In `app/ingestion/sentence_chunker.py`, every article is split into sentences, and each sentence gets an ID like D-01.S6, meaning article D-01, sentence 6. Then the sentences are packed into chunks of about 400 characters, with a one-sentence overlap, and the article title is added to each chunk's searchable text.

There are three reasons for this design. First, no text is lost. That fixes the starter-kit bug, and `tests/unit/test_chunking.py` proves every sentence ends up in at least one chunk. Second, we never cut a sentence in half, so a fact like "exports are capped at 10,000 rows" stays whole. Third, and most important, the LLM sees numbered sentences and must cite the IDs it used. Layer 3 can then check each ID against the sentences we actually sent, and check numbers against exactly those sentences. The UI also shows the admin the exact quoted sentence as the source.

I kept 400 characters because it is the starter kit's size and it works for these short help articles. The overlap means a fact that crosses a boundary still sits together with its neighbour in one chunk.

The limits are honest ones. Sentence splitting uses a simple rule: a full stop, question mark or exclamation mark followed by a capital letter, digit, quote or bracket. Abbreviations like "e.g." could split wrongly, but there are none in the seed docs. For larger or messier docs, I'd move to heading-aware chunking, so a chunk stays within one section.

Files to name: `app/ingestion/sentence_chunker.py`, `app/ingestion/corpus_builder.py`, `tests/unit/test_chunking.py`.

If time is left: With real docs I'd also store a URL and section anchor per chunk, so citations link to the live page. The seed data has no URLs, so today citations point to the article and sentence.

If they push back ("why not bigger chunks for more context?"): Bigger chunks make the cited context larger and the number check weaker. Small, precise chunks suit precision-first.

Q9. Do you even need an LLM? Your extractive mode got 25/25.

Rubric focus: Product Acumen, System Design.

Answer: Extractive mode got 25/25 on the dev set, which I tuned on. On the 13 held-out questions it got 9/13. That gap is the honest answer to this question.

Extractive mode never generates text. It returns the best-matching sentences, quoted exactly, so it cannot invent a fact. That is a real strength, and it is why I built it. It also costs nothing per question and answers in about 34 ms.

But it fails in two ways that matter for Elena. First, it relies on word overlap, so new vocabulary breaks it. "Minimum length" and "protocol" were refused even though the docs had the answer. Second, it can't judge meaning. When an admin asked how many members they can invite, a limit the docs don't state, it returned the nearest passage. It also answered 2 of 15 unanswerable questions in total, both in the held-out set. I added an entity check in `entity_coverage_checker.py` that marks an answer partial when the question names a plan, format or integration that the passage never mentions. That helps, but it is a rule, not real understanding.

LLM mode adds that judgement. It scored 42/42 in each of three runs, with zero unanswerable questions answered and zero false refusals. It also rewrites follow-ups, which extractive mode can't do.

So my recommendation is LLM mode, with extractive mode kept for two cases: a client who cannot send data to any LLM, and as the baseline that shows the LLM is worth paying for. Both modes run through the same strategy interface, `AnswerStrategy`, and switching is one setting: `LLM_PROVIDER=none`.

The extractive UI says "Most relevant passage, quoted from the docs", not "Answer", so the admin knows it is a quote.

Files to name: `app/services/answering/extractive_answer_strategy.py`, `app/services/answering/llm_answer_strategy.py`, `app/services/guards/entity_coverage_checker.py`, `eval/run_eval.py`.

If time is left: Mention cost. LLM mode adds roughly half a second and one small-model call per question. I did not build per-question cost tracking, and that is a known gap.

If they push back ("then delete extractive mode"): I'd keep it. It is the fallback for data-restricted clients, and it costs very little to keep.

Q10. Explain exactly what your grounding check verifies, and what it misses.

Rubric focus: Technical Execution, System Design.

Answer: Layer 3 lives in `app/services/guards/grounding_verifier.py`. It runs after the LLM answers and before anything is shown. It is plain code, so the same input always gives the same result.

It checks four things, and all four must pass. One: the answer is not empty. Two: there is at least one citation. Three: every cited sentence ID was in the context we actually sent the LLM. If the model cites D-99.S1, or a real sentence from an article we didn't send, the draft is rejected. Four: every number in the answer must appear in the cited sentences. Before comparing, commas inside numbers are removed, so "10,000" and "10000" match. If the LLM writes "links expire after 24 hours" but the cited sentence says 15 minutes, the draft is rejected.

If any check fails, the answer is discarded, the admin gets the honest refusal, and the `decision_reason` field says `grounding_check_failed` with the reason. Citation markers the model writes into the text are stripped before display, and the UI shows the real cited sentences instead.

I tested it with a scripted fake LLM in `tests/unit/test_answer_strategies.py`. The fake returns an invented "24 hours", an invented citation, or no citation. Each one is refused. These tests need no network or API key.

Now the honest blind spots. First, it checks citations and numbers, not every word. If the model changes "must" to "may", or says "you can" when the doc says "you cannot", this check won't catch it. Second, it compares digits only, so "two" or "twice" and units are not verified. Third, the "what is missing" text in a partial answer comes from the LLM and is not verified, because it describes an absence, not a fact.

The next step is an entailment check, also called NLI. A small model checks, sentence by sentence, whether the cited text actually supports each sentence of the answer. That extends Layer 3 from numbers to meaning. I would also check that key terms in the answer, like plan names, appear in the cited text.

Files to name: `app/services/guards/grounding_verifier.py`, `tests/unit/test_answer_strategies.py`, `tests/unit/test_guards_and_utils.py`.

If time is left: I chose numbers first because in support docs, numbers are where wrong answers hurt most: limits, timeouts, prices and rate limits.

If they push back ("so it's not real verification"): It is partial verification that is cheap and certain. It catches the most dangerous errors, and the gaps are documented.

Q11. Why no LangChain or vector database? Isn't that reinventing the wheel?

Rubric focus: System Design, Response to Pushback.

Answer: For this size and this requirement, plain code was the better choice, and I can say when I'd switch.

There are 12 chunks. A vector database adds a service to run, deploy and pay for, with no benefit at 12 chunks. Search over 12 chunks in memory takes milliseconds. So the corpus lives in memory behind `CorpusDAO`, in `app/dao/corpus_dao.py`.

LangChain gives fast building blocks, but it hides the parts that matter most here. The safety logic is the product: the relevance gate, the JSON status, the citation check and the number check. I want every step visible, testable, and easy to explain to Elena's team. With plain code, a reviewer can read `llm_answer_strategy.py` top to bottom and see each layer. It is also easier to test with a fake LLM, and there is one less dependency that changes often.

This is not a forever decision. The design is ready to grow. `CorpusDAO` is an interface, so Postgres with pgvector can be a new implementation once the docs reach the hundreds. The service code would not change. LLM providers sit behind a registry in `app/llm/llm_client_factory.py`, with Groq, Azure and OpenAI-compatible clients today.

I did reuse libraries where they don't hide logic: FastAPI, pydantic-settings, snowballstemmer for stemming, and sentence-transformers for embeddings. The one thing I wrote myself is BM25, in about 50 lines, so I could reuse its IDF weights.

When would I adopt LangChain or LlamaIndex? If the team already uses one of them as a standard, or if we needed many connectors quickly, like SharePoint, Confluence and Google Drive. Even then, I would keep the three layers as our own code.

Files to name: `app/dao/corpus_dao.py`, `app/llm/llm_client_factory.py`, `app/core/component_registry.py`, `app/services/answering/llm_answer_strategy.py`.

If time is left: In my day job I built a SharePoint SOP RAG chatbot, so I know where frameworks help, mainly with connectors and ingestion.

If they push back ("custom BM25 is risky"): It uses standard defaults, and the eval results show it works. `rank_bm25` is a one-line swap if the team prefers.

Q12. You built DTOs, mappers, DAOs and registries for a 3–4 hour prototype. Isn't that over-engineering?

Rubric focus: System Design, Communication, honesty.

Answer: It's a fair challenge, and I separated it clearly. In the design document I list core and extensions separately. The core is what the brief asked for: the retrieval fix, the three layers, the extractive fallback and the evaluation. The layered API, registries, central error handling and Docker are labelled as extensions.

Why I did the layering anyway: I wanted it to look like something a client team could take over and run, not a single script. In an FDE role, the code often stays with the client after we leave. Each layer has one job. Controllers in `app/api/controllers` only receive the request and wrap the response. `question_answering_service.py` holds the business flow. Strategies and guards decide and check the answer. DAOs handle where data lives. Every endpoint returns the same envelope, `ApiResponseDTO`, with a status, error code and request ID, so errors are consistent and easy to trace.

The practical benefits show in the code. Switching from Groq to Azure OpenAI is a config change, `LLM_PROVIDER`, not a code change. Switching to the no-LLM mode is `LLM_PROVIDER=none`. Moving to pgvector is a new `CorpusDAO` class. Tests can swap in a fake LLM easily, which is how 52 tests run with no network.

About time, I'll be honest. I specified the layered structure myself and used structured LLM prompting to scaffold the boilerplate quickly. That saved my own time for the parts that decide quality: retrieval, failure modes and the three layers. I said this in the design document too.

If the client wanted the smallest possible thing, I'd ship the core as a single FastAPI service file plus the guards. The three layers would stay exactly the same.

Files to name: `app/api/controllers/ask_controller.py`, `app/services/question_answering_service.py`, `app/core/component_registry.py`, `app/dto/api_response_dto.py`.

If time is left: The layering also keeps HTTP out of the services, so the same answering logic could run behind Slack or a batch job.

If they push back ("time was meant to be 3–4 hours"): Give your real number. The core took [your real hours]; extensions were extra and labelled. Don't guess a number you can't defend.

Q13. How do follow-up questions work, and how do you stop the conversation from making things up?

Rubric focus: System Design, Customer Mindset.

Answer: Follow-ups are handled in `app/services/followup_rewriter.py`, and the server stays stateless.

The UI sends the new question plus the last 3 turns. Earlier answers are cut to 300 characters to keep the prompt small. The server keeps no session, so any replica can handle any request, which keeps scaling simple.

If there is history, the LLM first rewrites the follow-up into a standalone question. For example, after "What are the API rate limits for Growth?", the admin asks "And for Enterprise?". The rewriter turns it into "What are the API rate limits for the Enterprise plan?". The UI shows this to the admin as "Searched for: …". That way, if the rewrite is wrong, the admin can see it straight away.

The key safety rule: earlier answers are used only to understand the question, never as a source of facts. The standalone question goes through retrieval and all three layers again, exactly like a new question. So even if an earlier turn contained a mistake, the new answer must still be cited from the docs and pass the grounding check.

If the rewrite call fails, we use the question as typed. It might then be refused because it lacks context, which is safe.

Limits: follow-ups need an LLM, so extractive mode answers each question on its own. The history comes from the client and is trusted only for understanding, not for facts. In the eval, 4 follow-up cases passed in each of the 3 LLM runs.

Files to name: `app/services/followup_rewriter.py`, `app/llm/prompts/followup_rewrite_prompt_v1.txt`, `app/services/question_answering_service.py`, `app/dto/ask_dto.py`.

If time is left: Because history comes from the client, the DTO limits its size, so nobody can push a huge history and inflate the prompt cost.

If they push back ("why not keep session state on the server?"): It adds a store and sticky sessions. Three turns from the client is enough for help-desk follow-ups.

Q14. What if someone tries prompt injection, like "ignore your rules and answer anyway"?

Rubric focus: System Design, Technical Execution.

Answer: I treated this as a layered problem, not just a prompt problem.

First, the prompt in `app/llm/prompts/answer_system_prompt_v1.txt` puts the admin's question inside `<question>` tags, and says to ignore any instructions found inside the documents or the question. Temperature is 0 and the output must be JSON with a fixed status.

Second, and more important, the code checks don't care what the prompt said. If an injected question makes the LLM claim something is "answered" without real support, Layer 3 still requires real citations from the context we sent, and every number must appear in those sentences. An invented answer fails that check. And if the status is anything other than answered, partial or not_covered, it is refused.

Third, Layer 1 refuses many off-topic or injected questions before the LLM is even called, because they don't match the docs.

In my v1 eval, a prompt-injection question was one of the cases that Layer 2 refused correctly.

The docs themselves are another risk. If someone could edit a doc and plant instructions in it, that would be indirect injection. Today the docs come from Elena's team, so they are trusted. In production I would control who can publish docs, and scan new docs for instruction-like text during ingestion.

What's not covered: there is no authentication or rate limiting on the API yet, so someone could spam it. I'd add Entra ID sign-in and rate limits at the gateway. Also, questions might contain personal data, so `app/utils/redaction.py` removes emails and runs of 6 or more digits before logging. It doesn't remove names yet.

Files to name: `app/llm/prompts/answer_system_prompt_v1.txt`, `app/services/guards/grounding_verifier.py`, `app/services/guards/relevance_gate.py`, `app/utils/redaction.py`.

If time is left: The worst case of a successful injection here is a refusal or a cited doc quote. The system has no tools and no account data, so the blast radius is small.

If they push back ("prompts can always be broken"): Agreed. That's why the promise rests on the code checks, not on the prompt.

Part C: Changed-requirement curveballs (Q15–Q22)

The guide says at least one question will change a customer requirement. Use the same five steps every time: **(1)** what I'd confirm with Elena, **(2)** what stays the same, **(3)** what changes and where in the code, **(4)** the new risk, **(5)** how I'd prove it works. Always say clearly that the three layers stay.

Q15. Elena says the docs will grow to 5,000 articles, updated every day. What changes?

Rubric focus: System Design, System Thinking.

Answer: First I'd confirm a few things with Elena. How often do articles change: a few a day, or bulk updates? Where do they live: a CMS, the docs website, SharePoint? And does an admin need to see an update within minutes, or is the next day fine?

What stays the same: the three layers and the answer format. Layer 1, Layer 2 and Layer 3 don't depend on corpus size. That is the point of keeping them as separate guards.

What changes is storage, ingestion and retrieval.

Storage: the in-memory index is rebuilt at every startup today. At 5,000 articles, roughly tens of thousands of chunks, I'd move to Postgres with pgvector. It holds the chunks, the vectors and full-text search in one place. This plugs in as a new `CorpusDAO` implementation in `app/dao/corpus_dao.py`, so the services don't change.

Ingestion: today `corpus_builder.py` runs inside the API at startup. I'd move it into a separate ingestion worker, triggered by a schedule or a webhook from the CMS. Each article gets a content hash, so we only re-chunk and re-embed articles that actually changed. The document source is already pluggable through

`DOCUMENT_SOURCE_REGISTRY`, so a CMS connector is a new class. For mixed formats, I'd use MarkItDown to turn PDF, Word or HTML into Markdown first, then use heading-aware chunking.

Retrieval: top 4 out of 12 chunks is a third of the corpus today. At this size, top 4 is a tiny slice, so recall matters more. I'd retrieve more candidates, say the top 30 from hybrid search, then add a cross-encoder reranker and pass the best 4 to 6 to the LLM. More articles also means more similar articles, like three pages about exports. The reranker and the citation check help there.

New risks: stale answers if ingestion fails silently, and thresholds tuned on 6 docs may no longer fit. So I'd monitor ingestion lag, re-run the eval after every bulk change, and re-tune thresholds on real questions.

Files to name: `app/dao/corpus_dao.py`, `app/ingestion/corpus_builder.py`, `app/dao/document_source_dao.py`, `app/retrieval/hybrid_retriever.py`.

If time is left: Store a doc version with each chunk. Then a logged answer can be traced to the exact version of the article it quoted.

Q16. The client says no data can leave their environment. Groq is not allowed. What now?

Rubric focus: Customer Mindset, System Design.

Answer: This is very common, and I planned for it. My design document assumes production runs on the client's own cloud, and Groq is only the demo provider.

First I'd clarify what "cannot leave" means. Is their own Azure tenant OK, or does it have to be fully on-premises with no cloud at all? The answer decides the option.

Option 1, their own cloud tenant, for example Azure OpenAI deployed in their subscription and region. This is a config change, not a code change. In `app/core/settings.py`, set `LLM_PROVIDER=azure` with their endpoint and key. The client lives in `app/llm/openai_compatible_llm_client.py`, and it is chosen through the registry in `llm_client_factory.py`. Secrets would go in Key Vault. Everything else stays the same.

Option 2, fully on-premises. Run an open-weight model inside their network behind an OpenAI-compatible server, such as vLLM. My OpenAI-compatible client works with any such endpoint, so again it's mostly configuration. Smaller local models are weaker at judgement, so I would re-run the full eval, all 42 questions plus held-out ones, before going live. Layer 3 matters even more here, because it protects us whatever model we plug in.

Option 3, no LLM at all. Set `LLM_PROVIDER=none` and run extractive mode. Nothing is generated, so no fact can be invented. But I'd be clear with Elena about the trade-off: 9 out of 13 on held-out questions versus 13 out of 13, and no follow-ups.

The embeddings already run locally with all-MiniLM-L6-v2, so search never sends data out in any option.

The log also matters. Questions are redacted for emails and long digit runs in `app/utils/redaction.py`, and the query log should stay inside their environment, with a retention policy.

Files to name: `app/core/settings.py`, `app/llm/llm_client_factory.py`, `app/llm/openai_compatible_llm_client.py`, `app/utils/redaction.py`.

If time is left: The LLM is called with only about four chunks of public help docs plus the question. The data actually sent is small, which can help a security review.

Q17. Elena now needs support for Spanish and Hindi questions. How does your design cope?

Rubric focus: System Thinking, Customer Mindset.

Answer: Today the system is English only, and I list that as a known limit. First I'd ask Elena two things. Are the docs also translated, or only the questions in other languages? And which languages come first, based on real ticket volume?

Case 1: questions in Spanish or Hindi, docs in English. Keyword search breaks first. BM25, my stopword list and the stemmer are all English. The fix is to translate the question to English first, using the LLM in the same step as the follow-up rewrite, then search as normal. The LLM can then write the answer in the admin's language. I'd also switch the embedding model to a multilingual one, so meaning search works across languages.

The tricky part is Layer 3. The citations stay the English source sentences, so the citation check still works. Numbers are digits in all of these languages, so the number check mostly works too. But the admin would see an answer in Hindi with an English source quote, and an error in translation is a new risk. I'd show both: the translated answer and the original English sentence.

Case 2: the docs are translated too. Then I'd index each language separately, tag each chunk with its language, and search in the admin's language. The guards work the same way, but the stopwords and stemming need per-language settings.

How I'd prove it: build eval questions per language from real tickets, and check that the refusal rate does not jump. A language that is poorly supported tends to show up as many false refusals.

Files to name: `app/retrieval/text_processing.py`, `app/services/followup_rewriter.py`, `app/retrieval/embedding_model.py`, `app/services/guards/grounding_verifier.py`.

If time is left: Extractive mode can't translate. For a non-English client without an LLM, it would only work if the docs exist in that language.

Q18. Admins now want personal answers, like "When is my renewal?" or "How many seats do I have left?"

Rubric focus: Customer Mindset, System Design, Product Acumen.

Answer: This changes the product. Today, my assumption is that answers come from the docs only, not account data. So "When is my renewal?" gets the general rule from the billing article, not the person's date. First I'd confirm with Elena that she really wants account data in the assistant. It brings security, privacy and testing work.

If yes, I'd keep two clearly separate sources: docs for how things work, and account data for this customer's facts.

Authentication becomes required first. Today the API has no auth. I'd add sign-in, for example Entra ID or Solstice's own login. The admin's company ID must come from their verified login, never from the question text. Otherwise someone could type "show me the seats for company X".

Then I'd add a small set of read-only account lookups, like "get renewal date" and "get seat count". The LLM can choose a lookup, but the code runs it with the logged-in company ID. That is a tool-calling design, with strict, typed tools.

The guarantee must extend too. Today, Layer 3 says every fact must come from cited doc sentences. With account data, every fact must come from either a cited doc sentence or a tool result. So the verifier would treat tool results as citable sources, for example a number like a seat count must match the tool result exactly.

The refusal rule stays. If a lookup fails or returns nothing, we say we can't confirm it, never a guess.

New risks: data leaking between customers, and personal data in logs. So add tests for cross-tenant access, extra redaction, and an audit log of every lookup.

Files to name: `app/services/question_answering_service.py`, `app/services/guards/grounding_verifier.py`, `app/core/middleware.py`, `app/utils/redaction.py`.

If time is left: I'd start with one lookup, like renewal date, measure how often it's asked, and only then add more. The gap report tells us which personal questions come up most.

Q19. Traffic grows 50 times and Elena wants answers in under one second. What breaks first?

Rubric focus: System Design, Technical Execution.

Answer: First, the facts I have. My assumption was tens to hundreds of questions a day. LLM mode answers with a p50 of 505 to 584 ms and a max of 1,215 ms on Groq. Extractive mode takes about 34 ms. So the median already fits one second, but the slow tail does not.

What breaks first at 50 times the traffic, in order:

One, the query log. It is a local JSONL file with a thread lock in `app/dao/query_log_dao.py`. That is safe for one process, not for several replicas. I'd move it to Postgres behind the same `QueryLogStore` interface, which is a registered component.

Two, LLM rate limits and cost. Groq's free tier has rate limits. In production we'd use Azure OpenAI with enough quota, and add a Redis cache for repeated questions. Elena said admins ask "the same handful of questions", so a cache keyed on the normalised standalone question plus the doc version could serve many answers with no LLM call. The cached answer was already verified once, and the cache entry is cleared when the doc version changes.

Three, a single process. The server is stateless, because follow-up history comes from the client, so I can run several replicas behind a load balancer on AKS or Container Apps. The corpus would need to move to pgvector or be loaded on each replica.

For the one-second target, I'd add a request time budget. Today the LLM timeout is 20 seconds, with one retry. I'd cap the total, for example at 2 seconds, and if it runs out, show the honest "can't verify right now" message rather than a slow answer. I'd also keep the prompt short, use a fast small model, and stream the answer to the UI.

What stays: all three layers. Layer 1 also helps here, because it refuses obvious misses without an LLM call.

How I'd prove it: a load test with real question patterns, tracking p50, p95, refusal rate and cost per question. Per-question cost tracking is a gap today that I'd add first.

Files to name: `app/dao/query_log_dao.py`, `app/core/component_registry.py`, `app/services/answering/llm_answer_strategy.py`, `app/core/settings.py`.

If time is left: A verified FAQ layer is the cheapest fast path. Approved answers for the top 20 questions, served instantly, with the LLM handling the long tail.

Q20. Elena changes her mind: "When the docs don't cover it, give your best guess with a warning." Do you do it?

Rubric focus: Customer Mindset, Response to Pushback.

Answer: I'd first understand why she wants it, because this reverses her original hard rule. Maybe admins are frustrated by too many refusals. Maybe a few specific topics are missing from the docs. The fix depends on the reason.

I'd show her the data first. The gap report at `/api/v1/insights/documentation-gaps`, from `insights_service.py`, lists the refused questions by frequency. If most refusals are a handful of missing topics, the best fix is to write those docs. Then the answers are real and verified, which is cheaper and safer than guessing.

If she still wants best-effort answers, I can build it safely, with clear rules.

One: keep it a separate, clearly labelled answer type, not mixed with verified answers. For example, the UI shows "Not in our documentation. General guidance, please verify" in a different style, with no citations. Today the response already has `answer_status` and `answer_type`, so this is a new type, not a hack.

Two: keep the three layers for verified answers exactly as they are. The best-effort path runs only after the system has already decided "not covered".

Three: block best-effort answers on risky topics, such as billing, security, data deletion and anything with numbers or limits. A wrong guess there causes real harm.

Four: make it a setting Elena controls, start with it off for most topics, and track thumbs-down feedback through `/api/v1/feedback`.

I'd be honest about the trade-off. The value of this tool is that admins can trust every answer. If guesses look too much like answers, that trust drops, and support tickets may go up, not down. So I'd recommend we pilot it with a small group first.

Files to name: `app/services/insights_service.py`, `app/dto/ask_dto.py`, `app/services/answering/llm_answer_strategy.py`, `app/services/feedback_service.py`.

If time is left: I'd write down her decision and the reason, so the change of rule is a documented client choice.

Q21. Some docs are only for Enterprise admins, and some are internal. How do you control access?

Rubric focus: System Design, Customer Mindset.

Answer: Today there is no access control. All docs are public help articles and the API has no auth, and I list both as gaps. If some docs are restricted, access must be enforced before retrieval, not after.

First, add sign-in, for example Entra ID, so we know the user's role and plan from a verified token, not from the question.

Second, add permission metadata to each document and chunk at ingestion: for example audience public or internal, and plans Growth or Enterprise. The `Document` and `Chunk` models in `app/models/corpus_models.py` would get these fields, filled by the document source.

Third, filter in retrieval. The hybrid retriever only searches chunks the user is allowed to see. In pgvector this is a simple WHERE filter on the same query. The important rule is that the LLM must never see a restricted chunk. If it never sees it, it can't leak it, whatever the prompt says.

The three layers stay the same. Layer 3 already rejects any citation that was not in the context given, so with filtered context it also blocks citing restricted text.

There is one subtle point about refusals. If a Growth admin asks about an Enterprise-only feature, should we say "not in the docs", or "this is available on Enterprise"? That is a product decision for Elena. Revealing that a feature exists might be fine, while revealing internal docs is not.

I'd test it with cases per role, checking that a Growth user never gets an Enterprise-only citation, and add these to the eval.

The idea is similar to my day job. In my RAG chatbot, access moved from a config file to central authorization tables, so the same pattern fits here.

Files to name: `app/models/corpus_models.py`, `app/retrieval/hybrid_retriever.py`, `app/dao/document_source_dao.py`, `app/core/middleware.py`.

If time is left: Also apply the same filter to the gap report and logs, so internal questions don't show up in a report for the wrong audience.

Q22. Two docs disagree: an old pricing page and a new one. Which answer does your system give?

Rubric focus: Debugging, System Thinking.

Answer: Honestly, today the system has no idea which one is newer. It would retrieve whichever chunk ranks highest. If both are in the top 4, the LLM sees both, and it might pick one, mix them, or mark the answer partial. Layer 3 would still pass, because both numbers really are in cited sentences. So this is a real gap. Layer 3 proves an answer is grounded in the docs, but not that the docs are correct or current.

How I'd fix it, starting with the cause.

One: add metadata at ingestion, such as last-updated date, version and a status like current or archived. Archived articles are excluded from search by default.

Two: add a conflict rule in the prompt and in the code. If the cited sentences contain different values for the same thing, the answer should not choose silently. It should either use the newest source and show its date, or return partial and say the docs disagree. A simple code check can help here: if two cited sentences from different articles have different numbers next to the same term, flag it.

Three: report it. A conflict is a documentation bug. I'd add conflicts to the gap report from `insights_service.py`, so Elena's team can fix the source, which is the real cure.

Four: show the article and date beside each citation in the UI, so the admin can judge for themselves.

The current seed docs have no dates or URLs, so I couldn't build this yet. That's why citations point to the article and sentence only.

Files to name: `app/models/corpus_models.py`, `app/ingestion/corpus_builder.py`, `app/services/guards/grounding_verifier.py`, `app/services/insights_service.py`.

If time is left: Content hashes and doc versions from Q15 also help. Each logged answer can be traced to the exact version it quoted, which settles disputes later.

Part D: Customer and business (Q23–Q26)

These questions test whether you think like someone working with the client, not just someone writing code. Talk about Elena, her admins, her team's time, and trust.

Q23. How would you know this is working once it's live? What would you measure?

Rubric focus: Product Acumen, Customer Mindset.

Answer: I'd measure three things: is it safe, is it useful, and is it reducing Elena's workload. Safety comes first, because of her rule.

Safety metrics. The most important number is wrong answers given confidently, which should be close to zero. I'd measure it by sampling answered questions every week and having a support person check them against the docs. I'd also track the thumbs-down rate from `/api/v1/feedback`, handled by `feedback_service.py`, and how often Layer 3 fires. The `decision_reason` field records `grounding_check_failed`, so a sudden jump tells us something changed, like a model update.

Usefulness metrics. The answer rate: how many questions get answered or partial, versus refused. Too many refusals means admins give up. The false-refusal rate: questions refused that the docs actually answer, found from weekly sampling. And latency, p50 and p95.

Workload metrics, which are what Elena actually cares about. The main one is the number of support tickets on topics the docs cover, compared before and after launch. If the assistant works, tickets about password resets and export limits should drop. I'd also track how many refused questions turn into tickets.

The gap report helps here too. `/api/v1/insights/documentation-gaps` lists the most common unanswered questions. When the docs team adds an article for the top gap, those questions should move from refused to answered. That is a clear, visible win to show Elena.

Operations: cost per question, LLM errors, and the "can't verify right now" rate. I haven't built per-question token and cost tracking yet. That's a known gap and the first thing I'd add, using OpenTelemetry to send it to App Insights.

I'd review these with Elena every week during the pilot, then monthly.

Files to name: `app/services/feedback_service.py`, `app/services/insights_service.py`, `app/dao/query_log_dao.py`, `app/core/middleware.py`.

If time is left: I'd agree targets with Elena before launch, for example zero confident wrong answers in the sampled set and a refusal rate she is comfortable with, so success is defined up front.

Q24. How would you explain the value and cost of this to Elena's manager?

Rubric focus: Product Acumen, Communication.

Answer: I'd keep it in their terms: support time saved, faster answers for admins, and low risk.

The value. Elena said new admins ask the same handful of questions that the docs already answer. Each of those tickets costs her team time to read, answer and close, and the admin waits. If the assistant answers them in about half a second with the exact quoted source, those tickets go away and the team works on harder problems. A second benefit is the gap report. It gives the docs team a ranked list of what's missing, based on what admins actually ask.

The cost. Infrastructure is one small service. LLM cost is one small-model call per question, with about four chunks of context. Layer 1 refuses obvious misses without any LLM call, and a cache would serve repeated questions for free. Honestly, I did not build per-question cost tracking, so I don't have a measured figure to quote. I'd add token logging first, run a pilot week, and give the manager a real cost per question and per month instead of a guess. I'd also say there is a zero-LLM-cost option, extractive mode, with lower quality: 9 out of 13 versus 13 out of 13 on held-out questions.

The risk. The usual fear with AI support tools is confident wrong answers. Here, refusal is enforced by code, not just by a prompt. Every number must appear in the quoted source, and if the LLM fails, it says so instead of guessing.

To make the business case real, I'd propose a pilot. Measure how many tickets on documented topics come in before and after, and multiply the drop by the average handling time. That's a number the manager can trust, because it comes from their own data.

Files to name: `app/services/insights_service.py`, `app/services/guards/relevance_gate.py`, `app/services/answering/extractive_answer_strategy.py`.

If time is left: Mention the low switching cost. The LLM provider is a setting, so they are not locked into one vendor's pricing.

Q25. How would you roll this out to Elena's admins?

Rubric focus: Customer Mindset, Product Acumen.

Answer: Gradually, with checks at each step, because trust is easy to lose and hard to win back.

Step 1, shadow mode, about one to two weeks. Before any admin sees it, I'd run real questions from Elena's ticket history through the system. Her team labels the correct answers, and we measure confident wrong answers, false refusals and partials. This replaces my self-written 42 questions with real ones, and it's the most important missing input. It becomes the permanent regression set, run with `eval/run_eval.py` in CI on every change.

Step 2, internal pilot. Elena's support agents use it first, as a helper while they answer tickets. They are experts, so they'll spot bad answers fast, and their thumbs-down feedback goes to `/api/v1/feedback`.

Step 3, limited admin pilot. A small group of customer admins, maybe new admins only, since they ask the most repeated questions. The UI keeps showing the quoted source and the "Searched for" line, so they can check for themselves.

Step 4, wider launch, with weekly reviews of the metrics and the gap report. The docs team fixes the top gaps each week.

Before any real users, I'd add the production basics I didn't build: authentication, rate limiting, Azure OpenAI in their tenant, a Postgres query log with a retention policy, and cost tracking.

At every step there is a clear way back. The LLM can be switched off with `LLM_PROVIDER=none`, which falls back to extractive quotes. Thresholds are settings, so they can be tightened without a deploy.

Files to name: `eval/run_eval.py`, `app/services/feedback_service.py`, `app/core/settings.py`, `app/static/index.html`.

If time is left: I'd also train the support team in five minutes on what "partial" and "not covered" mean, so they can explain it to admins.

Q26. What would you confirm with Elena before going further?

Rubric focus: Customer Mindset, Communication.

Answer: My design document lists my assumptions in a table, and each one is a question for Elena. I'd go through the ones with the biggest impact first.

One, the quality bar. I assumed precision over coverage, and that "partial" is a valid answer. I'd confirm she prefers "I don't know" over a mostly-right answer, and agree a target, like zero confident wrong answers in a weekly sample.

Two, the content. Where do the docs really live, how many are there, how often do they change, and who owns them? This decides the connector and re-indexing design.

Three, the users. Are they logged-in admins? Which languages? Do some docs apply only to certain plans? This decides auth and access filtering.

Four, the scope. I assumed answers come from docs only, not account data. So "when is my renewal" gets the general rule. I'd check that this is what she wants.

Five, the data and the LLM. Can questions go to an external LLM, or must it run in their own cloud? I assumed their own Azure tenant in production, with Groq for the demo only. Also, how long can we keep the question logs?

Six, the failure behaviour. If the LLM is down, I refuse with "can't verify right now". Is that acceptable, or does she want the extractive quote shown instead?

Seven, the data I most need: real questions from her ticket history. That replaces my self-written eval set.

I'd send a short written summary after the meeting, so the decisions are recorded and agreed.

Files to name: `app/core/settings.py`, `app/services/answering/llm_answer_strategy.py`, `data/eval_questions.json`.

If time is left: I'd also ask what a "wrong answer" has cost her in the past. That tells me how strict the thresholds should be.

Part E: Debugging, evaluation and honesty (Q27–Q30)

Debugging answers score well when they follow evidence step by step. Honesty answers score well when you give the gap plainly and then the plan.

Q27. An admin says the assistant gave a wrong answer. Walk me through how you debug it.

Rubric focus: Debugging, Customer Mindset.

Answer: First I'd thank the admin and get the facts: the exact question, roughly when it happened, and what the right answer should be. If they have the request ID, even better. Every response carries an `X-Request-ID`, added by `app/core/middleware.py`, and every log line has it.

Step 1, find the record. With the request ID I can find the log line, with latency and the decision, and the redacted query log entry in `query_log_dao.py`. The response itself shows the `answer_status`, the citations, the `standalone_question`, the `mode` and the `decision_reason`.

Step 2, find which stage failed. I'd ask four questions in order.

One: was the question understood? If it was a follow-up, check the "Searched for" rewrite. A bad rewrite in `followup_rewriter.py` sends everything after it in the wrong direction.

Two: was the right passage retrieved? I'd run the same standalone question through `hybrid_retriever.py` and look at the top 4 chunks. If the right sentence isn't there, it's a retrieval problem: a missing glossary synonym, a stopword issue, or the doc isn't indexed.

Three: did the LLM misread good evidence? If the right sentence was retrieved but the answer twists it, it's a Layer 2 problem. That is likely a wording error that Layer 3 can't catch, such as a dropped "not".

Four: is the doc itself wrong or out of date? Sometimes the system quotes correctly and the doc is the problem.

Step 3, reproduce and fix. I'd add the question to `data/eval_questions.json` as a failing case, then fix the cause: a glossary entry, a prompt rule, a threshold, or a doc update. I'd re-run `python -m eval.run_eval --runs 3` to confirm it's fixed and nothing else broke.

Step 4, close the loop. Tell the admin what happened and that it's fixed, and add the case to the regression set.

Files to name: `app/core/middleware.py`, `app/dao/query_log_dao.py`, `app/retrieval/hybrid_retriever.py`, `eval/run_eval.py`.

If time is left: If wrong answers are serious, a temporary safe move is tightening a threshold in settings. That makes the system refuse more until the fix ships, without a deploy.

Q28. Tell me about a real bug you found in your own work and how you fixed it.

Rubric focus: Debugging, honesty.

Answer: I'll give two, one about safety and one about retrieval.

The safety bug was in my v1. When the LLM call failed, the code fell back to extractive search and returned its result. On paper that sounded helpful. But in the v1 LLM eval, that fallback produced the only LLM-mode failure, case E-12. An answer was returned that the LLM had never judged. That breaks Elena's rule, because it is an unverified answer on a bad day. I found it because I added a mode column to the eval table. Before that, the table hid which path produced each answer.

The fix in v2, in `llm_answer_strategy.py`: retry the LLM once after 0.5 seconds, and if it still fails, return "I can't verify an answer right now". Authentication errors are not retried, because retrying won't fix a bad key. SDK retries are switched off so we don't retry twice. Extractive mode still exists, but only as a mode you choose, never as a hidden fallback.

The retrieval bug: "Is SSO available on the Growth plan?" retrieved the API article instead of the SSO one. I traced the scores and found the word "available" was matching the API article's text. In admin questions, "available" is just filler. I added it, with "solstice", as a domain stopword in the text processing step.

A third, smaller one: config precedence. A stale API key exported in my shell overrode my `.env` file, which caused auth errors. My first fix reversed the precedence, which then broke environment-based deployment. The final fix is the standard order, real environment first, then `.env`, then defaults, plus a startup log line showing where the key came from, masked.

What I learned: make each path visible in the eval and the logs. Most of these bugs were hidden by missing visibility, not by hard logic.

Files to name: `app/services/answering/llm_answer_strategy.py`, `app/retrieval/text_processing.py`, `app/core/settings.py`, `eval/run_eval.py`.

If time is left: Mention the Python 3.9 issue. I used 3.10-only syntax, and the sandbox ran 3.9, so I switched to `typing.Optional` and now test on both 3.9 and 3.12.

Q29. How did you evaluate, and why should I trust 42 out of 42?

Rubric focus: Technical Execution, Response to Pushback.

Answer: You shouldn't treat it as proof, and I don't. It is strong evidence that the design works on the cases I could think of.

How I evaluated. The script is `eval/run_eval.py` and the questions are in `data/eval_questions.json`. There are 25 dev questions, the 5 provided plus 20 I wrote, which I used while building and tuning. There are 13 held-out test questions, written before tuning and never tuned on. And there are 4 follow-up cases, for LLM mode only. Answerable and unanswerable questions are mixed, so the eval measures both wrong answers and false refusals.

A question passes only if three things hold: the status is one of the expected ones, an answered status cites the expected article, and the answer contains the key fact. That is stricter than the starter kit's eval, which only checked the document ID.

I ran LLM mode 3 times, because LLM output can vary. It got 42/42 every time, with 0 of 16 unanswerable answered and 0 of 26 false refusals. I also ran the starter kit with `--baseline` on the same questions. It got 4 out of 25 on dev and answered all 15 unanswerable ones.

Why not to over-trust it. First, I wrote the questions after reading the docs, so they're cleaner than real admin questions, which have typos and vague wording. Second, the set is small: 6 articles and 42 questions. Third, the judge checks key phrases, not full answer quality. Fourth, three runs is a small sample for variation.

The proof the risk is real: extractive mode dropped from 25/25 on dev to 9/13 on held-out. That's exactly why I kept a held-out set.

What would earn real trust: 100 to 200 real questions from Elena's tickets, labelled by her team, run in CI on every change and on every model change.

Files to name: `eval/run_eval.py`, `data/eval_questions.json`, `baseline/docs_qa_utils.py`.

If time is left: The eval writes a results file per mode, with a mode column and the answer text, so a human can read every answer, not just a score.

Q30. What did you not build, and what would you do with one more week?

Rubric focus: Communication, Product Acumen, honesty.

Answer: The guide asked me to say plainly what I didn't build, so here is the list.

Not built: authentication and rate limiting on the API. Per-question token and cost tracking. An overall request time budget. A persistent or versioned index: the corpus is rebuilt in memory at startup, with no live re-indexing. A production query log: today it's a local JSONL file, safe for one process only, with no retention policy. Multilingual support.

Citations to URLs, because the seed data has none. And Layer 3 checks citations and numbers but not full meaning. The 0.35 cosine threshold is still my least validated number. The cloud deployment is also missing. The image with embeddings was about 2.8 GB and needed more than 500 MB of memory, so free hosting didn't fit, and I run it locally with a tunnel for demos.

On time and AI use, I'll be direct. The core work, meaning the retrieval fix, the three layers, extractive mode and the eval, took [say your real hours]. The layered API, registries and Docker are extensions, labelled as such. I designed the structure myself and used structured LLM prompting to scaffold the boilerplate, which freed time for the parts that decide quality.

With one more week, in this order:

1. Real questions. Get 100 to 200 from Elena's tickets, label them with her team, and make that the regression set in CI. This is the most important step, because everything else is tuned against it.
2. Production basics: Entra ID auth, rate limits, Azure OpenAI in her tenant, a Postgres query log, and token and cost logging.
3. Quality: tune the cosine threshold on real data, and add an entailment check to extend Layer 3 beyond numbers.
4. Content: a connector to her real docs source, with re-indexing on change.

Files to name: `app/dao/query_log_dao.py`, `app/services/guards/grounding_verifier.py`, `app/dao/document_source_dao.py`, `eval/run_eval.py`.

If time is left: Close with the line that ties it all together: "The design doesn't need to change for any of this. The three layers stay, and each addition plugs into an interface that already exists."

Final 10-minute checklist

- [ ] Read the cheat sheet tables once, out loud.
- [ ] Fill in your real hours in Q12 and Q30, and practise saying them.
- [ ] Practise Q1 and one curveball (Q15 or Q16) with a timer, finishing at about 4 minutes.
- [ ] Close other apps and switch off notifications. A pop-up that takes focus counts as a tab switch.
- [ ] Desktop Chrome, Edge or Firefox; camera and mic allowed; share your entire screen.